

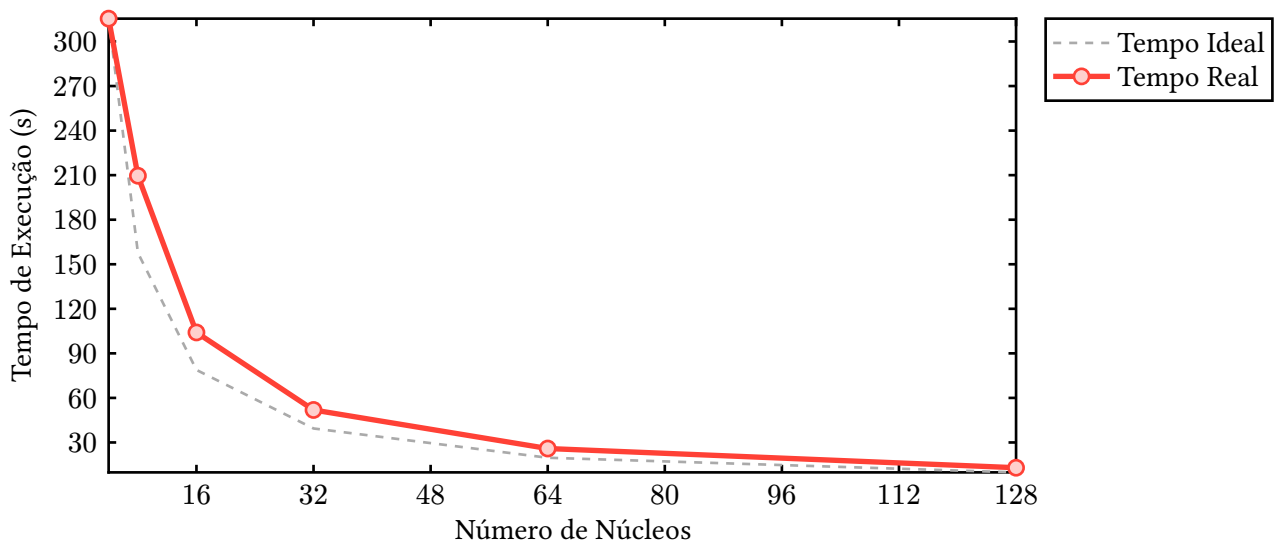
Escalonador Líder trabalhador

I. Metodologia

A implementação utiliza o padrão Líder-Trabalhador com escalonamento dinâmico. O processo líder é responsável por dividir a busca em lotes (chunks) e enviar uma tarefa inicial (contendo o início e fim de um intervalo) para cada trabalhador. À medida que um trabalhador conclui seu lote, ele devolve a contagem de primos encontrados. O líder recebe esse resultado parcial, soma ao total e imediatamente aloca um novo intervalo para esse trabalhador. Esse fluxo de distribuição sob demanda garante o balanceamento de carga, mantendo todos os processos ocupados até que o espaço de busca seja esgotado, momento em que o líder envia um sinal de encerramento aos trabalhadores.

II. Resultados

O gráfico abaixo ilustra o tempo de execução (em segundos) em função do número de núcleos utilizados. Para fins de comparação, também é apresentada a curva de tempo ideal (considerando o tempo inicial de 4 núcleos reduzindo perfeitamente pela metade a cada dobra na quantidade de processadores).



III. Anexo

```
#include <mpi.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

#define TAG_TASK 1
#define TAG_RESULT 2
#define TAG_KILL 3

typedef struct {
    int start;
    int end;
} Task;
```

```

typedef struct {
    int count;
} Result;

bool is_prime(int n) {
    if (n <= 1) {
        return false;
    }
    if (n <= 3) {
        return true;
    }

    if (n % 2 == 0 || n % 3 == 0) {
        return false;
    }

    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) {
            return false;
        }
    }
    return true;
}

void master_process(int world_size, int limit, int chunk_size) {
    int active_workers = world_size - 1;
    int current_val = 1;
    int total_primes = 0;

    MPI_Datatype mpi_task_type;
    MPI_Type_contiguous(2, MPI_INT, &mpi_task_type);
    MPI_Type_commit(&mpi_task_type);

    MPI_Datatype mpi_result_type;
    MPI_Type_contiguous(1, MPI_INT, &mpi_result_type);
    MPI_Type_commit(&mpi_result_type);

    // Distribuição inicial: envia uma tarefa para cada trabalhador
    for (int worker_id = 1; worker_id < world_size; worker_id++) {
        if (current_val <= limit) {
            Task t;
            t.start = current_val;
            t.end = (current_val + chunk_size - 1 > limit)
                ? limit
                : current_val + chunk_size - 1;

            MPI_Send(&t, 1, mpi_task_type, worker_id, TAG_TASK, MPI_COMM_WORLD);
            current_val += chunk_size;
        } else {
            // Se tivermos mais trabalhadores do que tarefas iniciais
            MPI_Send(NULL, 0, MPI_INT, worker_id, TAG_KILL, MPI_COMM_WORLD);
            active_workers--;
        }
    }
}

```

```

// 2. Escalonamento dinâmico: recebe resultado e envia nova tarefa
while (active_workers > 0) {
    Result res;
    MPI_Status status;

    // Recebe de qualquer trabalhador que terminar primeiro
    MPI_Recv(&res, 1, mpi_result_type, MPI_ANY_SOURCE, TAG_RESULT,
            MPI_COMM_WORLD, &status);
    total_primes += res.count;
    int worker_id = status.MPI_SOURCE;

    if (current_val <= limit) {
        Task t;
        t.start = current_val;
        t.end = (current_val + chunk_size - 1 > limit)
            ? limit
            : current_val + chunk_size - 1;

        MPI_Send(&t, 1, mpi_task_type, worker_id, TAG_TASK, MPI_COMM_WORLD);
        current_val += chunk_size;
    } else {
        // Acabaram as tarefas, manda o sinal de encerramento
        MPI_Send(NULL, 0, MPI_INT, worker_id, TAG_KILL, MPI_COMM_WORLD);
        active_workers--;
    }
}

printf("Lider finalizou. Total de primos encontrados: %d\n", total_primes);

MPI_Type_free(&mpi_task_type);
MPI_Type_free(&mpi_result_type);
}

void worker_process() {
    MPI_Datatype mpi_task_type;
    MPI_Type_contiguous(2, MPI_INT, &mpi_task_type);
    MPI_Type_commit(&mpi_task_type);

    MPI_Datatype mpi_result_type;
    MPI_Type_contiguous(1, MPI_INT, &mpi_result_type);
    MPI_Type_commit(&mpi_result_type);

    while (true) {
        MPI_Status status;

        // Verifica qual tipo de mensagem o lider mandou sem consumir da fila
        MPI_Probe(0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

        if (status.MPI_TAG == TAG_KILL) {
            // Consome a mensagem de kill e encerra o loop
            MPI_Recv(NULL, 0, MPI_INT, 0, TAG_KILL, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
            break;
        }
    }
}

```

```

Task t;
MPI_Recv(&t, 1, mpi_task_type, 0, TAG_TASK, MPI_COMM_WORLD,
        MPI_STATUS_IGNORE);

Result res;
res.count = 0;
for (int i = t.start; i <= t.end; i++) {
    if (is_prime(i)) {
        res.count++;
    }
}

MPI_Send(&res, 1, mpi_result_type, 0, TAG_RESULT, MPI_COMM_WORLD);
}

MPI_Type_free(&mpi_task_type);
MPI_Type_free(&mpi_result_type);
}

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, world_size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    if (rank == 0) {
        printf("Iniciando execução...\n");
    }

    if (world_size < 2) {
        if (rank == 0) {
            printf("Erro: Requer pelo menos 2 processos (1 lider, 1 trabalhador).\n");
        }
        MPI_Finalize();
        return 1;
    }

    // Parâmetros da simulação
    long long int limit = 1e9;
    long long int chunk_size = 10000; // 0 tamanho da tarefa (granularidade)

    MPI_Barrier(MPI_COMM_WORLD);
    double start_time = MPI_Wtime();

    if (rank == 0) {
        master_process(world_size, limit, chunk_size);
    } else {
        worker_process();
    }

    MPI_Barrier(MPI_COMM_WORLD);
    if (rank == 0) {
        double end_time = MPI_Wtime();
        printf("Tempo total: %f segundos\n", end_time - start_time);
    }
}

```

```
}  
  
MPI_Finalize();  
return 0;  
}
```